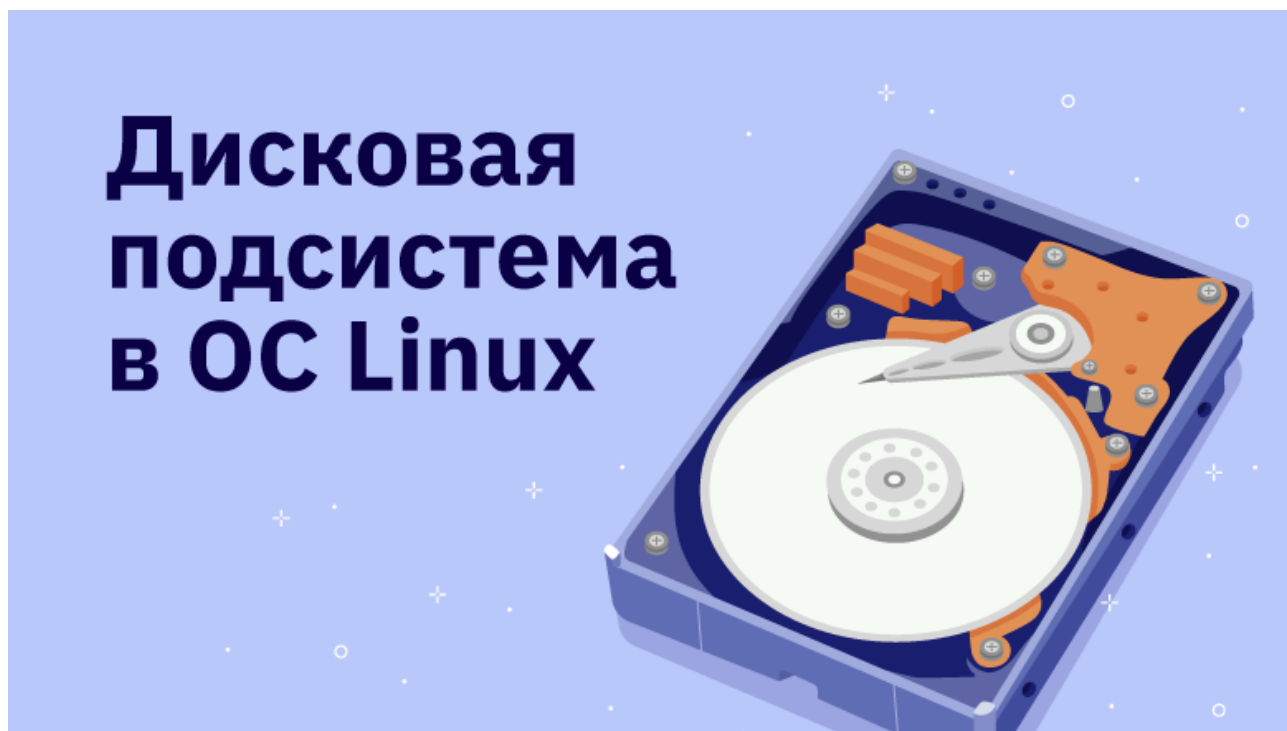


Дисковая подсистема в ОС Linux

 habr.com/ru/companies/otus/articles/713112

January 27, 2023

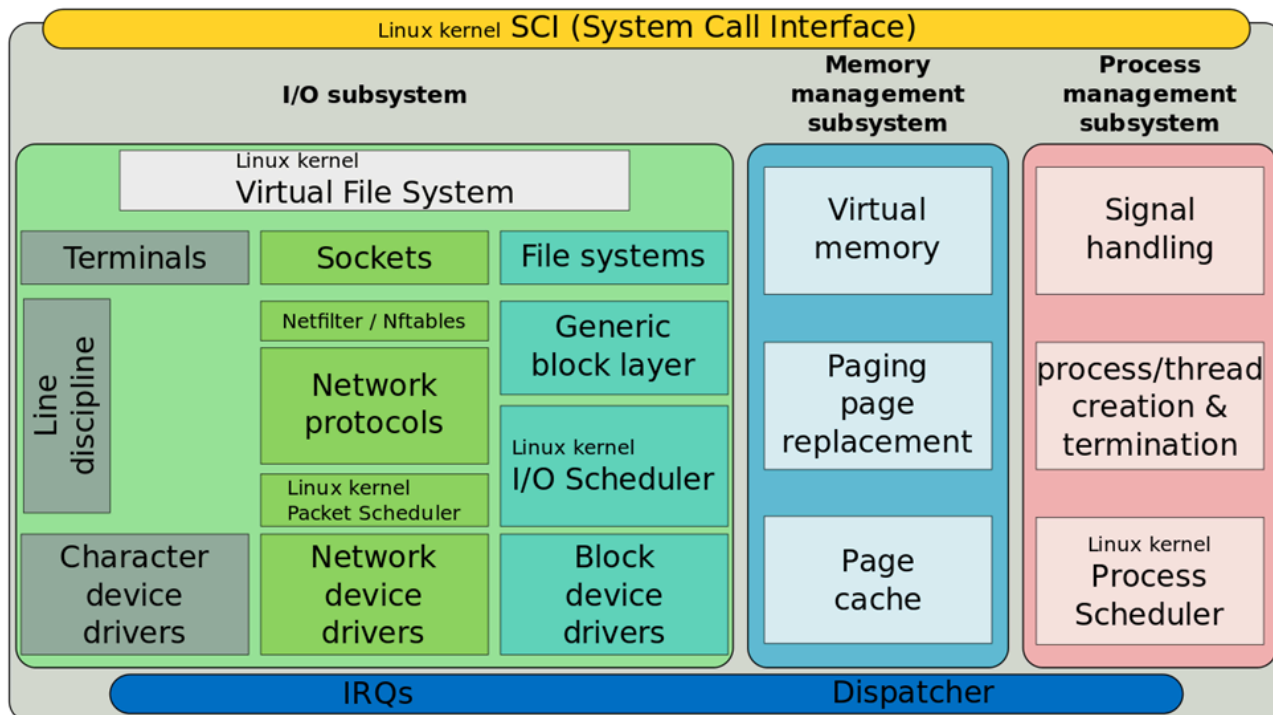


В сегодняшней статье мы поговорим об устройстве дисковой подсистемы в ОС Linux. Конечно, многие скажут, что на эту тему написано уже множество статей и все и так прекрасно знают, как устроен Линукс и в том числе, как он работает с дисками. Однако, как показывает практика, даже многие администраторы при работе с дисковой подсистемой ограничиваются только начальным разбиением диска при установке операционки. Да и то, с параметрами по умолчанию. Так что я предлагаю подробнее рассмотреть такие вопросы дисковой подсистемы, как: работу с правилами правила системы udev, инструменты для работы с блочными устройствами и планировщики системы ввода/вывода.

Только начинаете работать с Linux? [Подготовительный курс «Linux для начинающих»](#) поможет освоить командную строку, файловую систему, права доступа, процессы и службы.

Блочные и символьные

Начнем с рассмотрения файловой системы в Линуксе. На рисунке ниже представлены компоненты операционной системы Linux.



Помимо блочных, существуют также символьные (character device) устройства, которые используются для передачи данных побайтно, то есть без использования буфера. Здесь в качестве примеров можно упомянуть serial и parallel порт, звуковая карта, псевдоустройства `/dev/random`.

Посмотрим на практике, о чем идет речь:

```
cat /proc/devices
```

В начале выводятся символьные устройства:

```
root@ptus:~# cat /proc/devices
Character devices:
 1 mem
 4 /dev/vc/0
 4 tty
 4 ttyS
 5 /dev/tty
 5 /dev/console
 5 /dev/ptmx
 5 ttyprintk
```

Затем блочные:

```
135 sd
253 device-mapper
254 mdp
259 blkext
```

Также блочные устройства можно обнаружить с помощью `ls -l` и `ls -l`:

```
root@ptus:~# ls -l /dev/sda1
brw-rw---- 1 root disk 8, 1 Jan 25 12:27 /dev/sda1
```

```
root@ptus:~# lsblk
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0                               7:0    0   63.3M 1 loop /snap/core20/1778
loop1                               7:1    0    62M 1 loop /snap/core20/1587
loop3                               7:3    0   103M 1 loop /snap/lxd/23541
loop4                               7:4    0   49.8M 1 loop /snap/snapd/17950
loop5                               7:5    0  111.9M 1 loop /snap/lxd/24322
sda                                 8:0    0    20G 0 disk
├─sda1                             8:1    0     1M 0 part
├─sda2                             8:2    0    1.8G 0 part /boot
├─sda3                             8:3    0   18.2G 0 part
└─ubuntu--vg-ubuntu--lv 253:0    0    10G 0 lvm /
sr0                                 11:0    1    1.4G 0 rom
```

Обратите внимание на индексы во втором столбце вывода. Здесь 8 это старший номер (драйвер) устройства, а 1 — 3 это младшие номера. Старший номер 8 соответствует распределению номеров для SCSI-дисков, а младший номер — это номер раздела.

Для того, чтобы сопоставить то или иное блочное устройство его имени можно воспользоваться командой `blkid`:

```
root@ptus:~# blkid
/dev/sr0: BLOCK_SIZE="2048" UUID="2022-08-09-16-48-33-00" LABEL="Ubuntu-Server 2
2.04.1 LTS amd64" TYPE="iso9660" PTTYPE="PMBR"
/dev/mapper/ubuntu--vg-ubuntu--lv: UUID="70e62ed9-a3ec-477f-8ab7-0f89b4a714fa" B
LOCK_SIZE="4096" TYPE="ext4"
/dev/sda2: UUID="b35ce842-5747-480e-8ee1-b6ea95b840b3" BLOCK_SIZE="4096" TYPE="e
xt4" PARTUUID="c003c8ef-add6-49f8-8553-ab363329b673"
/dev/sda3: UUID="eeD7rY-Ctb0-Ycgc-g6GQ-8iss-ktQS-AutCtl" TYPE="LVM2_member" PART
UUID="a916afd9-8578-420a-b512-2c89aa0e477e"
/dev/loop1: TYPE="squashfs"
/dev/loop4: TYPE="squashfs"
/dev/loop0: TYPE="squashfs"
/dev/sda1: PARTUUID="0e12c502-6578-42dc-a382-a08ec87644a2"
/dev/loop5: TYPE="squashfs"
/dev/loop3: TYPE="squashfs"
```

При блочной передаче данных мы можем перемещаться по блоку данных, осуществлять в нем поиск и замену, кэшировать данные. При символьной передаче мы считываем только то, что нам передали.

Еще одна интересная команда для работы с контроллерами дисков это `lspci`.

```
root@ptus:~# lspci
00:00.0 Host bridge: Intel Corporation 440BX/ZX/DX - 82443BX/ZX/DX Host bridge (rev 01)
00:01.0 PCI bridge: Intel Corporation 440BX/ZX/DX - 82443BX/ZX/DX AGP bridge (rev 01)
00:07.0 ISA bridge: Intel Corporation 82371AB/EB/MB PIIX4 ISA (rev 08)
00:07.1 IDE interface: Intel Corporation 82371AB/EB/MB PIIX4 IDE (rev 01)
00:07.3 Bridge: Intel Corporation 82371AB/EB/MB PIIX4 ACPI (rev 08)
00:07.7 System peripheral: VMware Virtual Machine Communication Interface (rev 10)
00:0f.0 VGA compatible controller: VMware SVGA II Adapter
00:10.0 SCSI storage controller: Broadcom / LSI 53c1030 PCI-X Fusion-MPT Dual Ultra320 SCSI (rev 01)
00:11.0 PCI bridge: VMware PCI bridge (rev 02)
00:15.0 PCI bridge: VMware PCI Express Root Port (rev 01)
00:15.1 PCI bridge: VMware PCI Express Root Port (rev 01)
```

Данная команда выводит информацию по контроллерам дисков в различном виде. В выделенном блоке первое число это BUS, затем DEVICE и далее FUNCTION. То есть ШИНА, УСТРОЙСТВО и выполняемые ФУНКЦИИ.

С помощью ключей можно получить более подробную информацию по устройствам, представить данные в древовидной структуре, показать ID производителей.

```
# Оpciones команды
lspci -v -vv -vvv # больше деталей
lspci -tv # дерево соответствует физической структуре
lspci -D # показывает домен
lspci -nnA # показывает ID производителя
lspci -v -s 00:1f.2 # показать только одно устройство
```

Система Udev

Как мы помним, устройства в Линуксе это тоже файлы. Найти все файлы устройств можно в каталоге `/dev`. Содержимое этого каталога генерируется в процессе загрузки ОС специальным сервисом `udev`. Генерация этих файлов-устройств осуществляется по специальным правилам уже созданных в системе. Стоит отметить, что по умолчанию в Linux уже заложены все необходимые для нормальной работы устройств правила. Но иногда может возникнуть необходимость самостоятельно настроить правила для некоторых устройств. Например, с помощью правил `udev` можно разрешить подключение только определенных устройств, заблокировав все остальные, переименовать или создать дополнительное имя для устройства, поменять права доступа к устройству и так далее

Правила `udev` хранятся в папке `/etc/udev/rules.d`. В этой папке вы можете найти несколько правил, созданных ОС по умолчанию.

Далее в качестве примера мы создадим свое правило, которое будет давать флешке с определенным ID фиксированное имя каждый раз, при ее подключении.

Для начала нам необходимо узнать, какой идентификатор используется у данной флешки. Для начала, подключим флешку и посмотрим вывод lsblk:

```
lsblk
sdb          8:16    0    1G    0 disk
sdc          8:32    0    1G    0 disk
sdd          8:48    1 14.9G    0 disk
└─sdd1       8:49    1 14.9G    0 part
sr0         11:0    1   1.4G    0 rom
```

Теперь узнаем всю информацию для sdd1 с помощью команды:

```
udevadm info -n /dev/sdd1
```

Так как на экран будет выведено довольно много информации, можно сразу сделать выборку по ID_SERIAL

```
udevadm info -n /dev/sdd1 | grep ID_SERIAL
```

```
root@otus:/sys/block# udevadm info -n /dev/sdd1 | grep ID_SERIAL
E: ID_SERIAL=Kingston_DataTraveler_G3_0014780D6FC6FBA0A56B23B7-0:0
E: ID_SERIAL_SHORT=0014780D6FC6FBA0A56B23B7
```

Получаем значение идентификатора. Теперь перейдем к написанию правила Udev.

Скорее всего в папке с правилами у вас еще нет файла 69-disk.rules. Если это так, тогда создадим его:

```
nano /etc/udev/rules.d/69-disk.rules
```

Правило udev состоит из нескольких пар ключ — значение, разделенных запятой. Ключи используются для проверки соответствия устройства определённому правилу (например: SUBSYSTEM == «block» или ATTR{size}==«976773168»), для указания действия, если все условия соответствия выполняются (например: NAME=«mydisk» или SYMLINK+=«root»).

В нашем случае, содержимое файла будет следующим:

```
ACTION=="add", KERNEL=="sd[a-z]",
ENV{ID_SERIAL_SHORT}=="0014780D6FC6FBA0A56B23B7",
SYMLINK+="my_virtual_drive"
```

Далее сохраним изменения и проверим корректность написания правила:

```
udevadm test $(udevadm info --query=path --name=/dev/sda)
```

Далее добавим правило и перезагрузим правила в Udev.

```
udevadm control --reload-rules && udevadm trigger
```

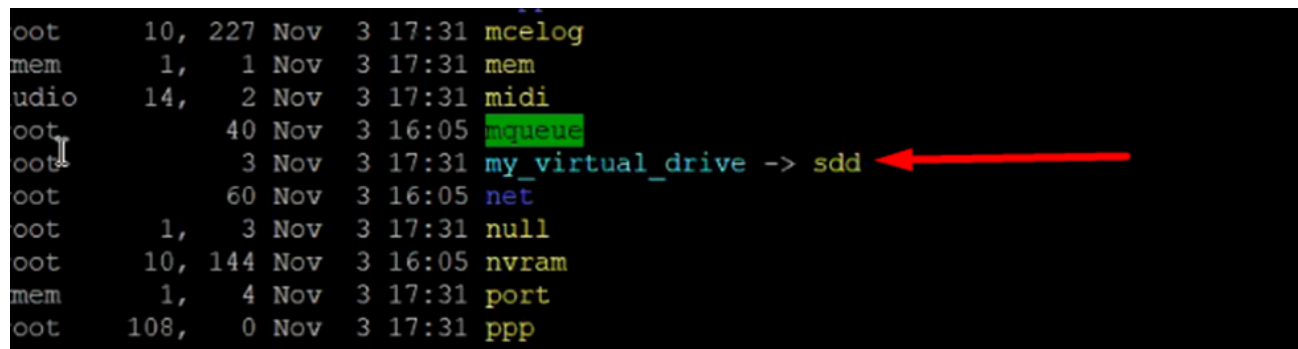
```
udevadm control --reload
```

```
sudo udevadm trigger --action=add
```

В результате при подключении флешки в выводе

```
ls -l /dev/
```

увидим следующее:



```
oot    10, 227 Nov  3 17:31 mcelog
mem     1,  1 Nov  3 17:31 mem
udio    14,  2 Nov  3 17:31 midi
oot     40, 40 Nov  3 16:05 queue
oot     3,  3 Nov  3 17:31 my_virtual_drive -> sdd
oot     60, 60 Nov  3 16:05 net
oot     1,  3 Nov  3 17:31 null
oot     10, 144 Nov  3 16:05 nvram
mem     1,  4 Nov  3 17:31 port
oot    108,  0 Nov  3 17:31 ppp
```

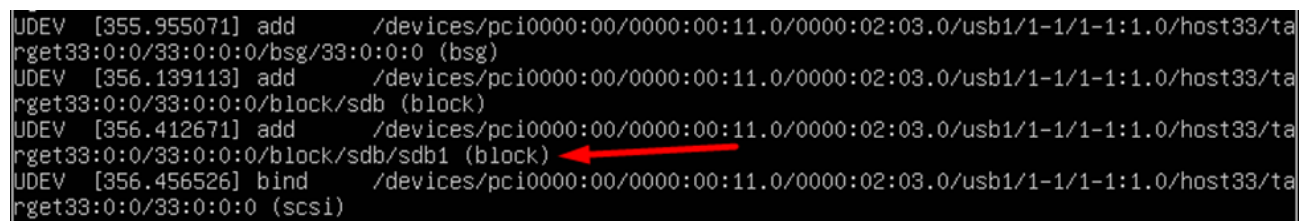
Но этот пример не слишком полезен в реальной жизни, только если у вас какие-то регулярно делают копирование на съемный носитель и вам важно, чтобы этот носитель всегда имел одно и то же имя.

А вот следующий пример может оказаться более полезным на практике. В реальной жизни флешки и USB-носители могут являться источником угроз, поэтому во многих организациях блокируют доступ к USB портам различными способами. Мы решим эту проблему следующим образом: разрешим подключение клавиатуры и мыши по USB, но запретим все остальное.

Для начала предлагаю познакомиться со средством мониторинга, командой `udevadm monitor`

```
udevadm monitor
```

Выполним эту команду и подключим флешку:



```
UDEV [355.955071] add /devices/pci0000:00/0000:00:11.0/0000:02:03.0/usb1/1-1/1-1:1.0/host33/target33:0:0/33:0:0:0/bsg/33:0:0:0 (bsg)
UDEV [356.139113] add /devices/pci0000:00/0000:00:11.0/0000:02:03.0/usb1/1-1/1-1:1.0/host33/target33:0:0/33:0:0:0/block/sdb (block)
UDEV [356.412671] add /devices/pci0000:00/0000:00:11.0/0000:02:03.0/usb1/1-1/1-1:1.0/host33/target33:0:0/33:0:0:0/block/sdb/sdb1 (block)
UDEV [356.456526] bind /devices/pci0000:00/0000:00:11.0/0000:02:03.0/usb1/1-1/1-1:1.0/host33/target33:0:0/33:0:0:0 (scsi)
```

На экране будет отображен процесс успешного подключения носителя. Теперь напишем само правило.

```
nano /etc/udev/rules.d/10-disable-usb.rules
```

Первой строкой мы все запрещаем (echo 0).

```
ACTION=="add", SUBSYSTEMS=="usb", RUN+="/bin/sh -c 'echo 0  
>/sys$DEVPATH/authorized'"
```

Второй строкой мы разрешаем (echo 1) те устройства, в названиях которых есть упоминание USB Optical Mouse|USB Keyboard.

```
ACTION=="add", SUBSYSTEMS=="usb", ATTR{product}=="USB Optical Mouse|USB  
Keyboard", RUN+="/bin/sh -c 'echo 1 >/sys$DEVPATH/authorized'"
```

Применим наши изменения:

```
udevadm control --reload-rules && udevadm trigger
```

```
udevadm control --reload
```

```
sudo udevadm trigger --action=add
```

Для того, чтобы убедиться в работе нового правила используем средства отладки Udev.

Снова воспользуемся командой:

```
udevadm monitor
```

После ее запуска подключаем флешку и смотрим на отображаемые события:

```
root@ptus:~# udevadm monitor
monitor will print the received events for:
UDEV - the event which udev sends out after rule processing
KERNEL - the kernel uevent

KERNEL[1117.643296] change   /devices/pci0000:00/0000:00:10.0/host16/target16:0:0/16:0:0:0/block/sda (block)
UDEV   [1117.752749] change   /devices/pci0000:00/0000:00:10.0/host16/target16:0:0/16:0:0:0/block/sda (block)
KERNEL[1127.654916] change   /devices/pci0000:00/0000:00:10.0/host16/target16:0:0/16:0:0:0/block/sda (block)
UDEV   [1127.745019] change   /devices/pci0000:00/0000:00:10.0/host16/target16:0:0/16:0:0:0/block/sda (block)
KERNEL[1137.666249] change   /devices/pci0000:00/0000:00:10.0/host16/target16:0:0/16:0:0:0/block/sda (block)
UDEV   [1137.765126] change   /devices/pci0000:00/0000:00:10.0/host16/target16:0:0/16:0:0:0/block/sda (block)
```

Событий, связанных с подключением sdb1 не зафиксировано как и подключенного USB носителя в системе.

Планировщики ввода-вывода

Еще один интересный компонент ОС Linux о которой не все знают это планировщики ввода-вывода (I/O Scheduling). Такое название получили методы управления

очередью операций ввода-вывода к жесткому диску и планировании данных операций. На практике с помощью планировщиков можно повысить производительность системы с помощью приоритезации некоторых запросов и увеличения производительности дисков.

В Linux на текущий момент существует четыре планировщика ввода-вывода:

- NOOP
- Anticipatory
- Deadline
- CFQ (Completely Fair Queuing — алгоритм полностью честной очереди)

NOOP

Планировщик NOOP получивший свое название от no operation на самом деле вовсе не является «бездельником», который не выполняет никаких операций. Он реализует простую очередь по принципу FIFO (First In First Out) первым вошел — первым вышел. NOOP не вносит изменений в порядок следования запросов к дискам, полагаясь на контроллеры самих дисков и RAID-контроллеров.

CFQ

Планировщик CFQ (Completely fair queuing) делит пропускную способность между всеми процессами. Для синхронных запросов создается по одной очереди на процесс, в то время как асинхронные запросы объединяются в очереди по приоритетам.

Deadline

У планировщика deadline использует за основу берется время нахождения запроса в очереди. Так как большинство приложений блокируются именно на чтении, то deadline по умолчанию отдает приоритет запросам на чтение.

Anticipatory

Планировщик anticipatory при работе предполагает, что следующий запрос будет блоку, идущему после текущего, только что обработанного. После выполнения операции чтения или записи планировщик делает некоторую задержку с тем, чтобы дать приложению время послать запрос на действие со следующим блоком. Если следующий запрос действительно окажется к следующему блоку, то головка диска будет уже находиться в нужном месте и не потребуется тратить время на ее перемещение. Такой метод удобен при работе с большими файлами, последовательно размещенными на диске.

Узнать какие планировщики доступны в вашем Линуксе можно с помощью команды:

```
cat /sys/block/sda/queue/scheduler
```

На моей Ubuntu доступны два варианта:

```
root@ptus:~# cat /sys/block/sda/queue/scheduler  
[mq-deadline] none
```

Для смены планировщика можно воспользоваться командой:

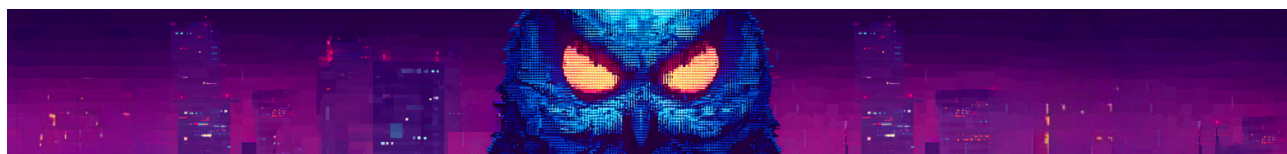
```
echo none >/sys/block/sda/queue/scheduler
```

```
root@ptus:~# cat /sys/block/sda/queue/scheduler  
[none] mq-deadline
```

Рекомендации по использованию тех или иных планировщиков можно поискать в Интернете, хотя здесь все возможно несколько субъективно и необходимо тестировать. Но в общем для NVMe дисков предлагается любой из многоочередных (deadline/bfq) или NOOP для уменьшения нагрузки на процессор. Для HDD deadline если у вас сервер и bfq если десктоп.

Заключение

В этой статье мы рассмотрели некоторые аспекты архитектуры и использования дисковой подсистемы ОС Linux. Однако, за кадром осталось еще много интересного: разделы, программные и аппаратные RAID и многое другое.



Хотите увереннее работать с Linux не только на уровне команд, но и на уровне системной архитектуры, устройств, служб и диагностики?

На курсе OTUS [«Администратор Linux. Продвинутый уровень»](#) разбираем работу с Linux-инфраструктурой глубже: от настройки системы и служб до диагностики, безопасности и отказоустойчивости.

Перед обучением можно пройти [бесплатное вступительное тестирование](#) — оно поможет оценить текущий уровень и понять, подходит ли вам программа.